Comparación experimental del rendimiento de SelectionSort y ShellSort según el tamaño y el patrón de entrada

Experimental Comparison of SelectionSort and ShellSort Performance According to Input Size and Pattern

Rodrigo Jhoel Llano Capia
rllanoc@unjbg.edu.pe
Ingeniería en Informática y Sistema
Universidad Nacional Jorge Basadre Grohmann
Tacna, Perú

Resumen*:*
Palabras claves:

Abstract:
Keywords: a

Introducción:

Los algoritmos de ordenamiento son fundamentales en la programación, ya que permiten organizar los datos y facilitar su búsqueda, procesamiento y análisis. Sin embargo, no todos los algoritmos presentan el mismo rendimiento, pues su eficiencia puede variar según la cantidad de elementos que deben ordenar y la distribución inicial de los datos.

Entre los algoritmos de ordenamiento se encuentran SelectionSort y ShellSort. El primero organiza los elementos mediante la selección sucesiva del menor valor de la parte no ordenada del arreglo. El segundo utiliza intervalos decrecientes para ordenar elementos que inicialmente pueden encontrarse alejados entre sí, hasta realizar una pasada final con un intervalo de uno. Estas diferencias en sus procedimientos permiten estudiar experimentalmente su comportamiento.

En este contexto, el presente estudio busca responder la siguiente pregunta: ¿cómo varían el tiempo de ejecución y la cantidad de operaciones de SelectionSort y ShellSort según el tamaño y el patrón de los datos de entrada?

El objetivo es comparar experimentalmente el rendimiento de ambos algoritmos mediante pruebas implementadas en C++, considerando diferentes tamaños de entrada y patrones de datos. Para ello, se medirán el tiempo de ejecución y las operaciones realizadas, con el propósito de describir sus diferencias y relacionarlas con sus características teóricas.

Como hipótesis descriptiva, se plantea que ShellSort podría presentar un menor tiempo de ejecución que SelectionSort a medida que aumenta el tamaño de la entrada. No obstante, el comportamiento observado dependerá del patrón de los datos y de la secuencia de intervalos utilizada por ShellSort.

2. Método

2.1. Algoritmos seleccionados

En el experimento se compararon dos algoritmos de ordenamiento: SelectionSort y ShellSort. Ambos se implementaron en el lenguaje C++ con el propósito de evaluar su rendimiento mediante pruebas repetibles.

SelectionSort. Este algoritmo recorre la parte no ordenada del arreglo para encontrar el elemento menor y colocarlo en su posición correspondiente. El procedimiento se repite hasta ordenar todos los elementos. Su complejidad temporal es $\Theta(n^2)$ en el mejor, promedio y peor caso, mientras que su espacio auxiliar es $O(1)$.

ShellSort. Este algoritmo ordena los elementos utilizando intervalos o incrementos que disminuyen progresivamente. En cada etapa compara elementos separados por el intervalo correspondiente y, finalmente, utiliza un intervalo de uno para completar el ordenamiento. Su rendimiento depende de la secuencia de incrementos utilizada; por ello, en este estudio se empleó la secuencia decreciente n/2, n/4, …, 1. Su espacio auxiliar es $O(1)$.

La secuencia de incrementos es importante para analizar ShellSort, ya que distintas secuencias pueden producir diferentes resultados de rendimiento (Shell, 1959).

2.2. Entorno de experimentación

Los algoritmos se implementaron en C++ y se utilizaron estructuras de arreglos dinámicos para almacenar los elementos. El tiempo de ejecución se midió mediante la biblioteca <chrono>, utilizando std::chrono::steady_clock, que permite medir intervalos de tiempo.

El programa se compiló con el estándar C++17 y la opción de optimización -O2. Para garantizar la reproducibilidad, se registraron los parámetros de cada ejecución y la semilla utilizada para generar los datos aleatorios.

Nota: en la versión definitiva se especificarán el procesador, la memoria RAM, el sistema operativo y la versión exacta del compilador utilizados durante las pruebas.

2.3. Diseño experimental y patrones de entrada

Se consideraron cuatro patrones de entrada: datos aleatorios, datos ordenados de forma ascendente, datos ordenados de forma descendente y datos con valores duplicados. Estos patrones permiten observar el comportamiento de los algoritmos ante diferentes distribuciones iniciales.

Se establecieron inicialmente tamaños de entrada de 1 000 y 3 000 elementos, con 30 repeticiones por combinación de algoritmo, tamaño y patrón. Los tamaños se eligieron como punto de partida para realizar pruebas controladas, considerando que SelectionSort presenta un crecimiento cuadrático de sus operaciones.

En cada repetición, ambos algoritmos recibieron una copia de los mismos datos iniciales. La generación de los datos y la copia del arreglo se realizaron fuera del intervalo cronometrado para medir principalmente el tiempo dedicado al ordenamiento.

2.4. Métricas de evaluación

Se consideraron las siguientes métricas:

- **Tiempo de ejecución:** duración del ordenamiento, expresada en milisegundos.
- **Comparaciones:** cantidad de comparaciones entre elementos realizadas por cada algoritmo, según el contador implementado.
- **Intercambios y movimientos:** operaciones utilizadas para cambiar de posición o desplazar elementos durante el ordenamiento.
- **Memoria auxiliar:** se consideró el espacio adicional requerido por cada algoritmo, sin incluir el arreglo de entrada.
- **Correctitud:** se verificó que el resultado estuviera ordenado y conservara los elementos originales.

Debido a que ambos algoritmos son iterativos, la profundidad de recursión no se consideró una métrica aplicable.

2.5. Procedimiento y reproducibilidad

Para cada combinación de algoritmo, tamaño y patrón se realizaron 30 repeticiones. Se registraron los resultados individuales en un archivo CSV y se calcularon estadísticas descriptivas, incluyendo media, mediana, desviación estándar, mínimo y máximo.

La comparación se realizará utilizando las mismas entradas para ambos algoritmos. Los datos obtenidos permitirán elaborar tablas y gráficos para estudiar las diferencias de rendimiento. El código fuente, los parámetros experimentales y los archivos de resultados se organizarán en un repositorio de GitHub para facilitar la revisión y reproducción del experimento.

3. Resultados: